Prueba de Endpoints y Guarantee Event Consume Order en CQRS y Event Sourcing

Spring Boot · Apache Kafka · MongoDB · MySQL

Estudiante: Martín Gómez Caicedo

Código: A00399958

Repositorio base: jrquinte/cqrs-event-sourcing-kafka

Fork de trabajo: github.com/Electromayonaise/cqrs-event-sourcing-kafka

Fecha: 20 de octubre de 2026

Contenido

1. Alcance y correspondencia con el enunciado 3
2. Entorno de ejecución 3
3. Parte 1: Prueba de endpoints 4
   3.1 OpenAccountController: abrir cuenta 5
   3.2 DepositFundsController: depositar fondos 6
   3.3 WithdrawFundsController: retirar fondos 6
   3.4 AccountLookupController: consultas (GET) 7
   3.5 CloseAccountController: cerrar cuenta 9
   3.6 RestoreReadDbController: restaurar la base de datos de lectura 10
   3.7 Flujo de eventos observado en los logs 11
4. Parte 2: Cambios para garantizar el orden de consumo 12
   4.1 El problema 12
   4.2 Cambios solicitados por el enunciado 12
   4.3 Cambios complementarios 16
5. Parte 2: Prueba de reproducción de eventos 17
   5.1 Un solo topic en Kafka 17
   5.2 Serie de operaciones 17
   5.3 Reinicio simple del servicio de consulta 19
   5.4 Reproducción completa de los eventos 19
   5.5 Resultado: el modelo de lectura reconstruido es idéntico 21
6. Cómo se garantiza el orden y por qué es importante 23
   6.1 Cómo se garantiza 23
   6.2 Por qué es importante en CQRS y Event Sourcing 23

1. Alcance y correspondencia con el enunciado

Este documento reúne los tres entregables de la tarea. La siguiente tabla indica dónde se cubre cada punto del enunciado:

Requisito del enunciado	**Sección**
Parte 1: probar OpenAccount, DepositFunds, WithdrawFunds, CloseAccount, AccountLookup (4 GET) y RestoreReadDb, con URL, método, cuerpo y captura de la respuesta	3.1 a 3.6
Parte 2.1: un solo topic en application.yml; AccountEventStore y AccountEventSourcingHandler con el nuevo topic; EventConsumer y AccountEventConsumer con un solo método	4.2
Parte 2.2: serie de operaciones (abrir, depositar, retirar, cerrar) y reinicio del servicio de consulta	5.2 a 5.4
Parte 2.3: capturas de logs con el orden de consumo y explicación de cómo se garantiza el orden y por qué importa	5.2, 5.4 y 6
Entregable 1: documento con capturas y explicaciones de cada endpoint	3
Entregable 2: documento con los cambios para garantizar el orden de consumo	4
Entregable 3: capturas y explicación de la prueba de reproducción de eventos	5

El enunciado remite al documento “Guarantee Event Consume Order.pdf”, que no está incluido en el repositorio; los cambios se implementaron siguiendo los pasos que el propio enunciado enumera (topic único *BankAccountEvents*, publicación en ese topic y consumo con un solo método). Las pruebas de ambas partes se ejecutaron sobre la versión final del código, que ya incluye los cambios de la Parte 2; la interfaz HTTP de los controllers no cambia con esos ajustes.

2. Entorno de ejecución

El sistema está compuesto por dos microservicios Spring Boot que aplican CQRS: **account.cmd** (lado de escritura, puerto 5000), que valida los comandos en el agregado *AccountAggregate*, persiste los eventos en el event store (MongoDB) y los publica en Kafka; y **account.query** (lado de lectura, puerto 5001), que consume los eventos de Kafka y mantiene el modelo de lectura en MySQL.

Para ejecutar todo de forma reproducible se agregó un *docker-compose.yml* que levanta Zookeeper, Kafka, MongoDB y MySQL y construye ambos microservicios con un *Dockerfile* multi-etapa. Las pruebas se ejecutaron en Windows con Docker Desktop mediante scripts de PowerShell que llaman cada endpoint con *curl.exe* y guardan la solicitud, la respuesta y los logs; las capturas se generaron con Playwright a partir de esa evidencia.

Contenedor	**Imagen**	**Puerto**	**Rol**
zookeeper	`confluentinc/cp-zookeeper:7.6.1`	2181	Coordinación de Kafka
kafka	`confluentinc/cp-kafka:7.6.1`	29092	Bus de eventos
mongo	`mongo:4.4`	27017	Event store (escritura)
mysql	`mysql:8.0`	3306	Base de datos de lectura
account-cmd	`techbank/account-cmd`	5000	Command API

Contenedor	Imagen	Puerto	Rol
account-query	techbank/account-query	5001	Query API

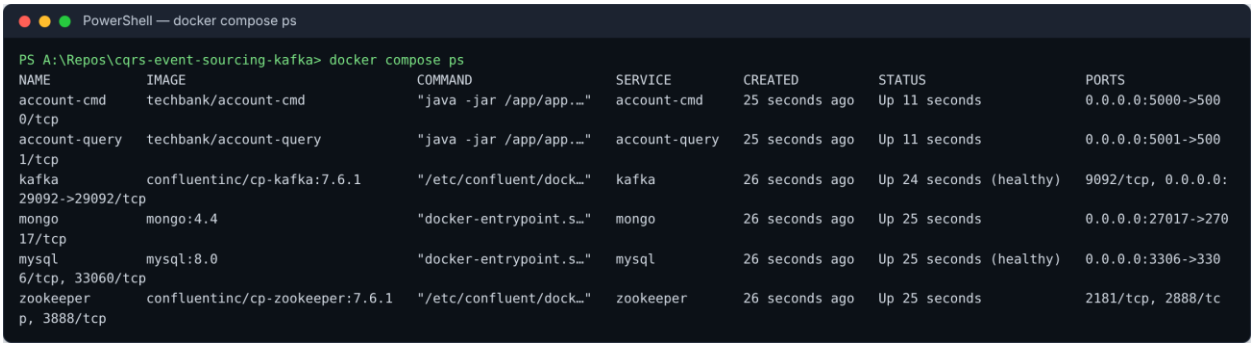


Figura 1. *Contenedores en ejecución (docker compose ps).*

Versión de Java. El proyecto usa Spring Boot 2.4.5, que trae Lombok 1.18.20, incompatible con JDK 17 o superior. Sin modificar los *pom.xml*, el *Dockerfile* compila con JDK 17 (imagen oficial *eclipse-temurin*) y fuerza una versión reciente de Lombok solo en ese build (*-Dlombok.version=1.18.34*).

Horas. El campo "Ejecutado" de cada captura de respuesta está en hora local (Colombia, UTC-5); los logs de los contenedores están en UTC.

3. Parte 1: Prueba de endpoints

Se ejecutó un flujo completo sobre dos cuentas: la cuenta **A** (Juan Perez, SAVINGS, saldo inicial 1000) recibe un depósito, un retiro y finalmente se cierra; la cuenta **B** (Maria Gomez, CURRENT, saldo inicial 5000) permanece activa para que las consultas tengan más de un resultado. Las consultas se hicieron antes de cerrar la cuenta A para ver ambas cuentas en el modelo de lectura.

#	Controller	Método	Endpoint	HTTP
1	OpenAccountController	POST	`/api/v1/openBankAccount`	201
2	DepositFundsController	PUT	`/api/v1/depositFunds/{id}`	200
3	WithdrawFundsController	PUT	`/api/v1/withdrawFunds/{id}`	200 / 400
4	CloseAccountController	DELETE	`/api/v1/closeBankAccount/{id}`	200
5a	AccountLookupController	GET	`/api/v1/bankAccountLookup/`	200
5b	AccountLookupController	GET	`/api/v1/bankAccountLookup/byId/{id}`	200 / 204
5c	AccountLookupController	GET	`/api/v1/bankAccountLookup/byHolder/{holder}`	200
5d	AccountLookupController	GET	`/api/v1/bankAccountLookup/withBalance/{tipo}/{balance}`	200
6	RestoreReadDbController	POST	`/api/v1/restoreReadDb`	201

Los endpoints de comandos responden en *http://localhost:5000* y los de consulta en *http://localhost:5001*.

3.1 OpenAccountController: abrir cuenta

Método	POST
URL	`http://localhost:5000/api/v1/openBankAccount`
Cuerpo	`{"accountHolder": "Juan Perez", "accountType": "SAVINGS", "openingBalance": 1000}`

El controller genera un UUID, despacha un *OpenAccountCommand* y responde **201 Created** con el ID de la cuenta: *96ab93b2-ece5-4188-8a95-b46cac3fb1e1*. Este ID se usa en las operaciones siguientes sobre la cuenta A.

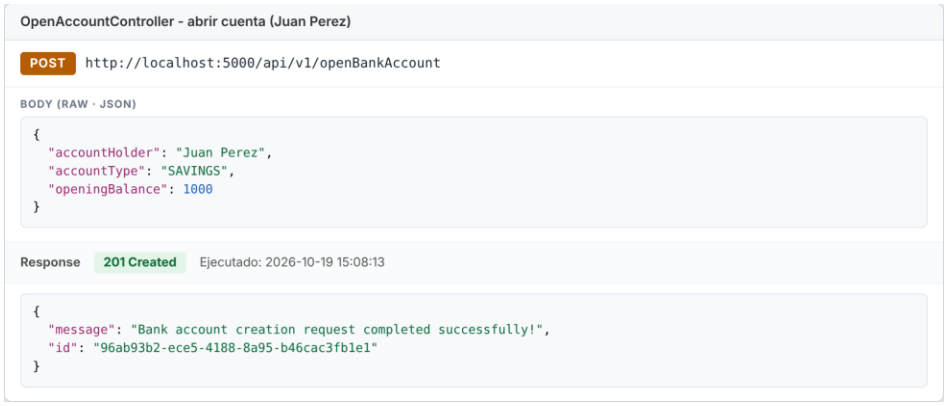


Figura 2. *POST openBankAccount: cuenta A creada, con su ID en la respuesta.*

Con el mismo endpoint se abrió la cuenta B (titular Maria Gomez, tipo CURRENT, saldo inicial 5000; el cuerpo se ve en la Figura 3), que obtuvo el ID *1e8a4ba8-56cb-4310-a093-1024c9567c9a*.



Figura 3. *POST openBankAccount: cuenta B creada.*

3.2 DepositFundsController: depositar fondos

Método	PUT
URL	`http://localhost:5000/api/v1/depositFunds/96ab93b2-ece5-4188-8a95-b46cac3fb1e1`
Cuerpo	`{"amount": 500}`

El agregado valida que la cuenta esté activa y que el monto sea positivo, y genera un *FundsDepositedEvent*. Saldo esperado de A: 1000 + 500 = 1500.



Figura 4. *PUT depositFunds: depósito de 500 en la cuenta A.*

3.3 WithdrawFundsController: retirar fondos

Método	PUT
URL	`http://localhost:5000/api/v1/withdrawFunds/96ab93b2-ece5-4188-8a95-b46cac3fb1e1`
Cuerpo	`{"amount": 200}`

Genera un *FundsWithdrawnEvent*. Saldo esperado de A: 1500 − 200 = 1300.



Figura 5. *PUT withdrawFunds: retiro de 200 de la cuenta A.*

También se probó la regla de fondos insuficientes: un retiro por 99999 es rechazado con **400 Bad Request** y no genera ningún evento.



Figura 6. *PUT withdrawFunds con monto mayor al saldo: rechazado con 400.*

3.4 AccountLookupController: consultas (GET)

Las consultas leen el modelo de lectura en MySQL, que se actualiza de forma asíncrona al consumir los eventos (consistencia eventual); por eso el script espera unos segundos después de los comandos antes de consultar. Ninguna consulta GET lleva cuerpo.

a. Obtener todas las cuentas

Método	GET
URL	`http://localhost:5001/api/v1/bankAccountLookup/`
Cuerpo	No aplica

Devuelve ambas cuentas. La cuenta A muestra **balance 1300.0**: el depósito y el retiro llegaron al modelo de lectura en el orden correcto.

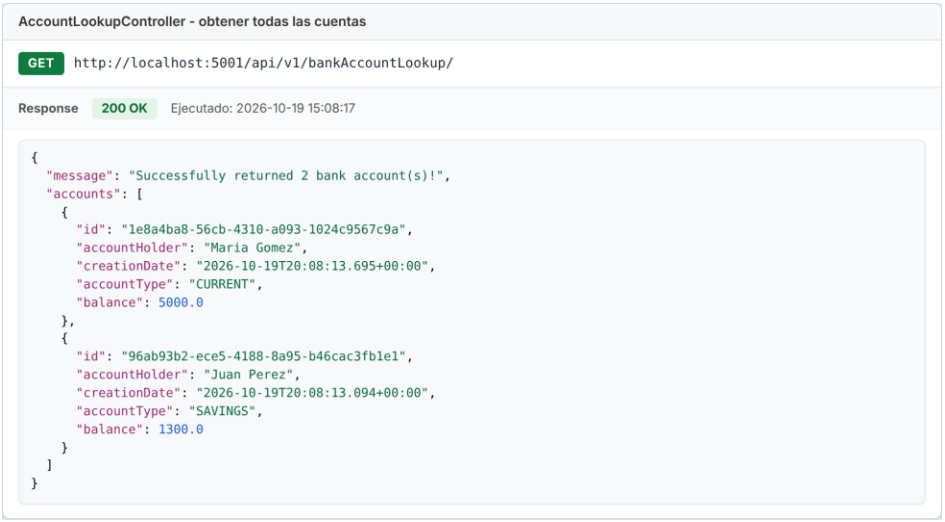


Figura 7. *GET bankAccountLookup/: todas las cuentas.*

b. Buscar cuenta por ID

Método	GET
URL	`http://localhost:5001/api/v1/bankAccountLookup/byId/96ab93b2-ece5-4188-8a95-b46cac3fb1e1`
Cuerpo	No aplica

Devuelve únicamente la cuenta A.



Figura 8. *GET byId: cuenta A por su ID.*

c. Buscar cuenta por titular

Método	GET
URL	`http://localhost:5001/api/v1/bankAccountLookup/byHolder/Maria%20Gomez`
Cuerpo	No aplica

El nombre del titular va codificado en la URL (el espacio se envía como %20).



Figura 9. *GET byHolder: cuenta de Maria Gomez.*

d. Buscar cuentas con un balance específico

El enum *EqualityType* del proyecto solo define *GREATER_THAN* y *LESS_THAN* (el README menciona *EQUAL_TO*, pero no está implementado), así que se probaron ambos valores con el umbral 2000.

Método	GET
URL	`http://localhost:5001/api/v1/bankAccountLookup/withBalance/GREATER_THAN/2000`
Cuerpo	No aplica

Devuelve solo la cuenta B (5000 > 2000).

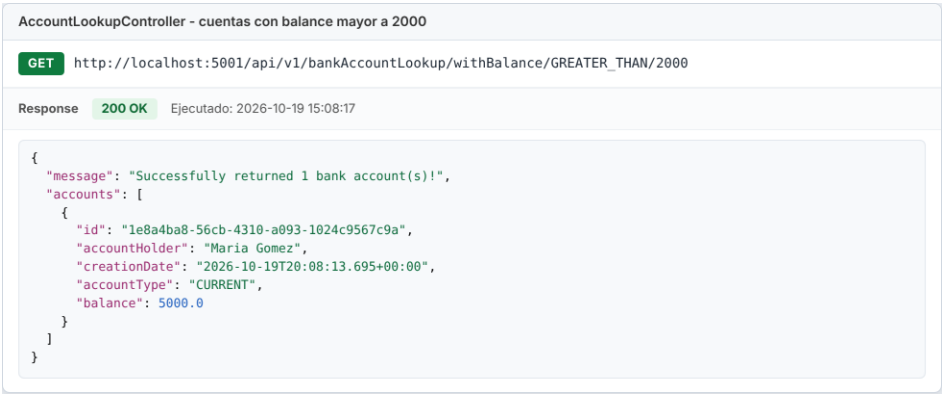


Figura 10. *GET withBalance/GREATER_THAN/2000.*

Método	GET
URL	`http://localhost:5001/api/v1/bankAccountLookup/withBalance/LESS_THAN/2000`
Cuerpo	No aplica

Devuelve solo la cuenta A (1300 < 2000).

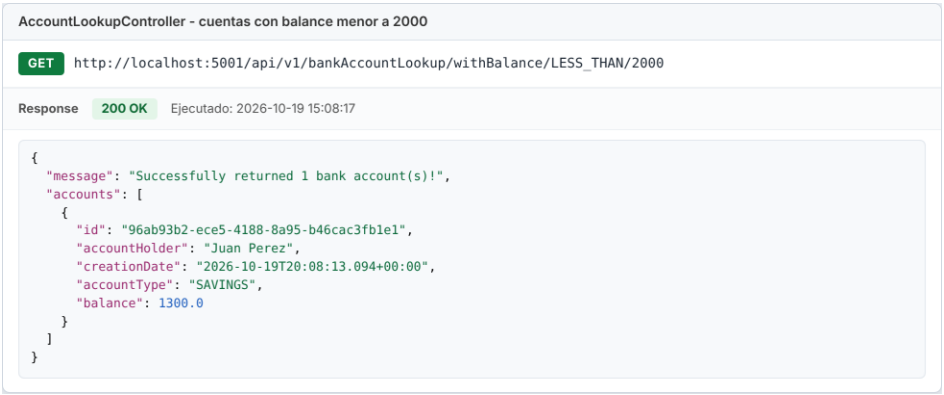


Figura 11. *GET withBalance/LESS_THAN/2000.*

3.5 CloseAccountController: cerrar cuenta

Método	DELETE
URL	`http://localhost:5000/api/v1/closeBankAccount/96ab93b2-ece5-4188-8a95-b46cac3fb1e1`
Cuerpo	No aplica

Genera un *AccountClosedEvent*; al consumirlo, el lado de lectura elimina la cuenta del modelo de lectura.



Figura 12. *DELETE closeBankAccount: cierre de la cuenta A.*

Verificación: al buscar la cuenta A por ID después del cierre, el servicio responde **204 No Content**, y el listado completo ya solo contiene la cuenta B.

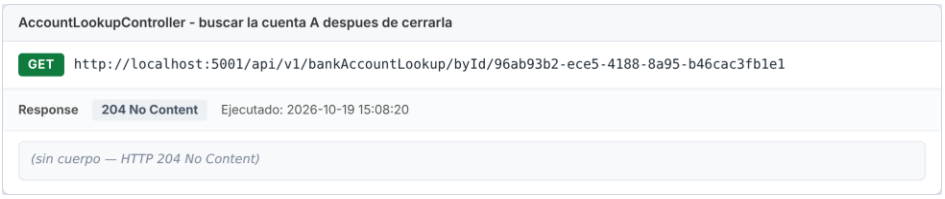


Figura 13. *GET byId de la cuenta A después del cierre: 204 No Content.*

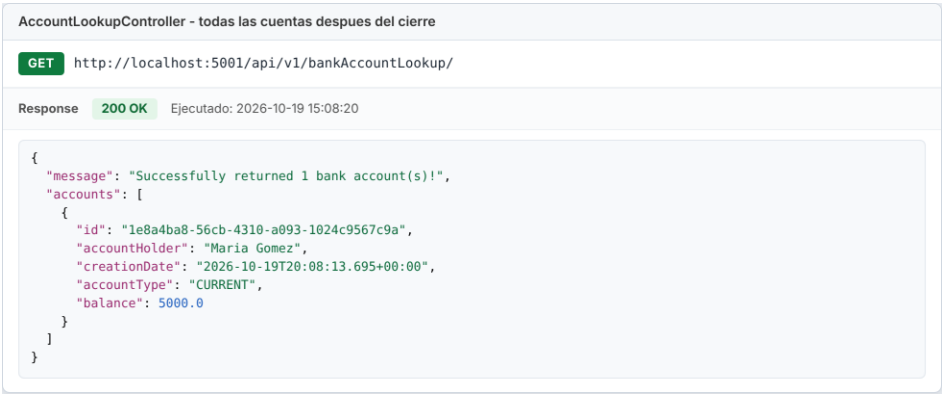


Figura 14. *GET de todas las cuentas después del cierre: solo queda la cuenta B.*

3.6 RestoreReadDbController: restaurar la base de datos de lectura

Método	POST
URL	`http://localhost:5000/api/v1/restoreReadDb`
Cuerpo	No aplica

Despacha un *RestoreReadDbCommand*: *AccountEventSourcingHandler.republishEvents()* lee el event store (MongoDB) y vuelve a publicar en Kafka todos los eventos de las cuentas **activas**, para que el lado de lectura reconstruya su estado. Las cuentas cerradas se omiten, por eso solo se republicó el *AccountOpenedEvent* de la cuenta B (offset 5 del topic, visible en la Figura 22).

RestoreReadDbController - restaurar la base de datos de lectura

POST http://localhost:5000/api/v1/restoreReadDb

Response 201 Created Ejecutado: 2026-10-19 15:08:20


```
{
  "message": "Read database restore request completed successfully!"
}
```


Figura 15. *POST restoreReadDb: 201 Created.*

AccountLookupController - todas las cuentas despues del restore

GET http://localhost:5001/api/v1/bankAccountLookup/

Response 200 OK Ejecutado: 2026-10-19 15:08:23


```
{
  "message": "Successfully returned 1 bank account(s)!",
  "accounts": [
    {
      "id": "1e8a4ba8-56cb-4310-a093-1024c9567c9a",
      "accountHolder": "Maria Gomez",
      "creationDate": "2026-10-19T20:08:13.695+00:00",
      "accountType": "CURRENT",
      "balance": 5000.0
    }
  ]
}
```


Figura 16. *GET de todas las cuentas después del restore: el modelo de lectura sigue consistente.*

3.7 Flujo de eventos observado en los logs

Cada comando exitoso produjo un evento en el topic *BankAccountEvents* (líneas PRODUCED del servicio de comandos) y el servicio de consulta lo consumió en el mismo orden (líneas CONSUMED, offsets 0 a 5).

PowerShell — logs de los microservicios durante la Parte 1 (líneas de eventos)

```
PS A:\Repos\cqrs-event-sourcing-kafka> docker logs account-cmd    # líneas PRODUCED
2026-10-19 20:08:13.587  INFO 1 --- [nio-5000-exec-3] c.t.a.c.i.AccountEventProducer          : PRODUCED  topic=BankAccountEvents key=96ab93b2-ece5-418
8-8a95-b46cac3fb1e1 type=AccountOpenedEvent version=0
2026-10-19 20:08:13.702  INFO 1 --- [nio-5000-exec-5] c.t.a.c.i.AccountEventProducer          : PRODUCED  topic=BankAccountEvents key=1e8a4ba8-56cb-431
0-a093-1024c9567c9a type=AccountOpenedEvent version=0
2026-10-19 20:08:13.816  INFO 1 --- [nio-5000-exec-7] c.t.a.c.i.AccountEventProducer          : PRODUCED  topic=BankAccountEvents key=96ab93b2-ece5-418
8-8a95-b46cac3fb1e1 type=FundsDepositedEvent version=1
2026-10-19 20:08:13.899  INFO 1 --- [nio-5000-exec-9] c.t.a.c.i.AccountEventProducer          : PRODUCED  topic=BankAccountEvents key=96ab93b2-ece5-418
8-8a95-b46cac3fb1e1 type=FundsWithdrawnEvent version=2
2026-10-19 20:08:17.522  INFO 1 --- [nio-5000-exec-2] c.t.a.c.i.AccountEventProducer          : PRODUCED  topic=BankAccountEvents key=96ab93b2-ece5-418
8-8a95-b46cac3fb1e1 type=AccountClosedEvent version=3
2026-10-19 20:08:20.733  INFO 1 --- [nio-5000-exec-4] c.t.a.c.i.AccountEventProducer          : PRODUCED  topic=BankAccountEvents key=1e8a4ba8-56cb-431
0-a093-1024c9567c9a type=AccountOpenedEvent version=0
PS A:\Repos\cqrs-event-sourcing-kafka> docker logs account-query  # líneas CONSUMED
2026-10-19 20:08:13.694  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer        : CONSUMED  partition=0 offset=0 type=AccountOpenedEvent
id=96ab93b2-ece5-4188-8a95-b46cac3fb1e1 version=0
2026-10-19 20:08:13.781  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer        : CONSUMED  partition=0 offset=1 type=AccountOpenedEvent
id=1e8a4ba8-56cb-4310-a093-1024c9567c9a version=0
2026-10-19 20:08:13.825  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer        : CONSUMED  partition=0 offset=2 type=FundsDepositedEvent
id=96ab93b2-ece5-4188-8a95-b46cac3fb1e1 version=1
2026-10-19 20:08:13.908  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer        : CONSUMED  partition=0 offset=3 type=FundsWithdrawnEvent
id=96ab93b2-ece5-4188-8a95-b46cac3fb1e1 version=2
2026-10-19 20:08:17.531  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer        : CONSUMED  partition=0 offset=4 type=AccountClosedEvent
id=96ab93b2-ece5-4188-8a95-b46cac3fb1e1 version=3
2026-10-19 20:08:20.741  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer        : CONSUMED  partition=0 offset=5 type=AccountOpenedEvent
id=1e8a4ba8-56cb-4310-a093-1024c9567c9a version=0
```

Figura 17. *Logs de producción y consumo de eventos durante la Parte 1.*

4. Parte 2: Cambios para garantizar el orden de consumo

4.1 El problema

En el código original cada tipo de evento se publicaba en un topic propio, cuyo nombre era el de la clase (*AccountOpenedEvent*, *FundsDepositedEvent*, *FundsWithdrawnEvent*, *AccountClosedEvent*), y el servicio de consulta tenía un *@KafkaListener* por topic. Kafka garantiza el orden **solo dentro de una partición**; entre topics distintos no existe orden relativo, y cada listener corre en su propio hilo consumidor.

En operación normal, como los eventos se generan con segundos de diferencia, el problema casi no se nota. Pero al reproducir los eventos (un servicio de consulta nuevo, una base de lectura reconstruida, offsets reiniciados), los cuatro listeners leen en paralelo y el modelo de lectura puede recibir, por ejemplo, un *FundsDepositedEvent* antes del *AccountOpenedEvent* de la misma cuenta: el handler no encuentra la cuenta y el depósito se pierde. El modelo de lectura queda inconsistente con el event store.

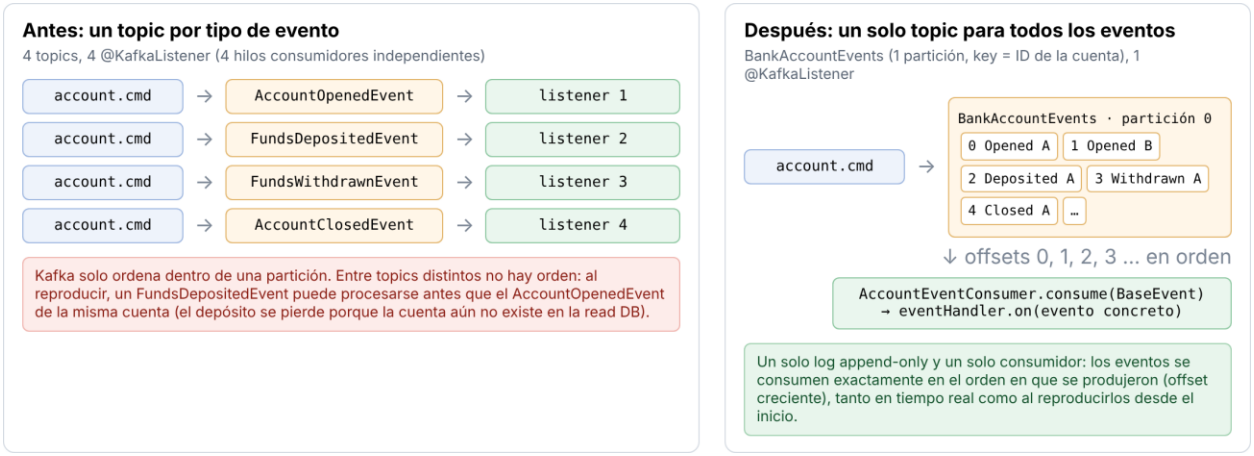


Figura 18. *Antes y después: de un topic por tipo de evento a un único topic ordenado.*

4.2 Cambios solicitados por el enunciado

a. application.yml: un solo topic

Se agregó la propiedad *spring.kafka.topic* con el nombre del topic único, en ambos servicios:

```
account.cmd/src/main/resources/application.yml
@@ -9,4 +9,6 @@ spring:
       database: bankAccount
   kafka:
+    # Guarantee Event Consume Order: un solo topic para todos los eventos del agregado
+    topic: BankAccountEvents
     producer:
       bootstrap-servers: localhost:29092
account.query/src/main/resources/application.yml
@@ -15,4 +15,6 @@ spring:
       password: techbankRootPsw
   kafka:
+    # Guarantee Event Consume Order: un solo topic para todos los eventos del agregado
+    topic: BankAccountEvents
     listener:
       ack-mode: MANUAL_IMMEDIATE
```

b. AccountEventStore: publicar en el topic único

Al persistir cada evento en MongoDB, ahora se publica en el topic configurado en lugar del topic con el nombre de la clase:

```
account.cmd/.../infrastructure/AccountEventStore.java
@@ -10,4 +10,5 @@ import com.techbank.cqrs.core.infrastructure.EventStore;
 import com.techbank.cqrs.core.producers.EventProducer;
 import org.springframework.beans.factory.annotation.Autowired;
+import org.springframework.beans.factory.annotation.Value;
 import org.springframework.stereotype.Service;

@@ -24,4 +25,8 @@ public class AccountEventStore implements EventStore {
     private EventStoreRepository eventStoreRepository;

+    // Topic unico para todos los eventos (Guarantee Event Consume Order)
+    @Value("${spring.kafka.topic}")
+    private String topic;
+
     @Override
     public void saveEvents(String aggregateId, Iterable<BaseEvent> events, int expectedVersion) {
@@ -44,5 +49,5 @@ public class AccountEventStore implements EventStore {
             var persistedEvent = eventStoreRepository.save(eventModel);
             if (!persistedEvent.getId().isEmpty()) {
-                eventProducer.produce(event.getClass().getSimpleName(), event);
+                eventProducer.produce(topic, event);
             }
         }
```

c. AccountEventSourcingHandler: republicar en el topic único

El mismo cambio en *republishEvents()*, usado por el endpoint de restauración de la base de lectura:

```
account.cmd/.../infrastructure/AccountEventSourcingHandler.java
@@ -7,4 +7,5 @@ import com.techbank.cqrs.core.infrastructure.EventStore;
 import com.techbank.cqrs.core.producers.EventProducer;
 import org.springframework.beans.factory.annotation.Autowired;
+import org.springframework.beans.factory.annotation.Value;
 import org.springframework.stereotype.Service;

@@ -19,4 +20,8 @@ public class AccountEventSourcingHandler implements EventSourcingHandler<Account
     private EventProducer eventProducer;

+    // Topic unico para todos los eventos (Guarantee Event Consume Order)
+    @Value("${spring.kafka.topic}")
+    private String topic;
+
     @Override
     public void save(AggregateRoot aggregate) {
@@ -45,5 +50,5 @@ public class AccountEventSourcingHandler implements EventSourcingHandler<Account
             var events = eventStore.getEvents(aggregateId);
             for(var event: events) {
-                eventProducer.produce(event.getClass().getSimpleName(), event);
+                eventProducer.produce(topic, event);
             }
         }
```

d. EventConsumer: una sola firma para todos los eventos

La interfaz pasa de cuatro métodos (uno por tipo de evento) a uno solo que recibe *BaseEvent*. Se agregan la partición y el offset como parámetros para registrarlos en el log como evidencia del orden.

EventConsumer.java (antes)

```
package com.techbank.account.query.infrastructure.consumers;

import com.techbank.account.common.events.AccountClosedEvent;
import com.techbank.account.common.events.AccountOpenedEvent;
import com.techbank.account.common.events.FundsDepositedEvent;
import com.techbank.account.common.events.FundsWithdrawnEvent;
import org.springframework.kafka.support.Acknowledgment;
import org.springframework.messaging.handler.annotation.Payload;

public interface EventConsumer {
    void consume(@Payload AccountOpenedEvent event, Acknowledgment ack);
    void consume(@Payload FundsDepositedEvent event, Acknowledgment ack);
    void consume(@Payload FundsWithdrawnEvent event, Acknowledgment ack);
    void consume(@Payload AccountClosedEvent event, Acknowledgment ack);
}
```

EventConsumer.java (después)

```
package com.techbank.account.query.infrastructure.consumers;

import com.techbank.cqrs.core.events.BaseEvent;
import org.springframework.kafka.support.Acknowledgment;
import org.springframework.kafka.support.KafkaHeaders;
import org.springframework.messaging.handler.annotation.Header;
import org.springframework.messaging.handler.annotation.Payload;

/**
 * Guarantee Event Consume Order:
 * en lugar de un metodo (y un topic) por tipo de evento, un solo metodo consume
 * todos los eventos del topic unico, en el mismo orden en que fueron producidos.
 */
public interface EventConsumer {
    void consume(@Payload BaseEvent event,
                 @Header(KafkaHeaders.RECEIVED_PARTITION_ID) int partition,
                 @Header(KafkaHeaders.OFFSET) long offset,
                 Acknowledgment ack);
}
```

e. AccountEventConsumer: un único listener

Un único *@KafkaListener* escucha el topic configurado. El *JsonDeserializer* reconstruye la clase concreta del evento a partir del header *__TypeId__* que agrega el *JsonSerializer* del productor; luego se despacha por reflexión al método *on(...)* del *EventHandler* que corresponde a esa clase. El offset se confirma (*ack.acknowledge()*) solo después de aplicar el evento.

AccountEventConsumer.java (antes)

```
package com.techbank.account.query.infrastructure.consumers;

import com.techbank.account.common.events.AccountClosedEvent;
import com.techbank.account.common.events.AccountOpenedEvent;
import com.techbank.account.common.events.FundsDepositedEvent;
import com.techbank.account.common.events.FundsWithdrawnEvent;
```

```
import com.techbank.account.query.infrastructure.handlers.EventHandler;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.support.Acknowledgment;
import org.springframework.messaging.handler.annotation.Payload;
import org.springframework.stereotype.Service;

@Service
public class AccountEventConsumer implements EventConsumer {
    @Autowired
    private EventHandler eventHandler;

    @KafkaListener(topics = "AccountOpenedEvent", groupId = "${spring.kafka.consumer.group-id}")
    @Override
    public void consume(@Payload AccountOpenedEvent event, Acknowledgment ack) {
        this.eventHandler.on(event);
        ack.acknowledge();
    }

    @KafkaListener(topics = "FundsDepositedEvent", groupId = "${spring.kafka.consumer.group-id}")
    @Override
    public void consume(@Payload FundsDepositedEvent event, Acknowledgment ack) {
        this.eventHandler.on(event);
        ack.acknowledge();
    }

    @KafkaListener(topics = "FundsWithdrawnEvent", groupId = "${spring.kafka.consumer.group-id}")
    @Override
    public void consume(@Payload FundsWithdrawnEvent event, Acknowledgment ack) {
        this.eventHandler.on(event);
        ack.acknowledge();
    }

    @KafkaListener(topics = "AccountClosedEvent", groupId = "${spring.kafka.consumer.group-id}")
    @Override
    public void consume(@Payload AccountClosedEvent event, Acknowledgment ack) {
        this.eventHandler.on(event);
        ack.acknowledge();
    }
}
```

AccountEventConsumer.java (después)

```
package com.techbank.account.query.infrastructure.consumers;

import com.techbank.account.query.infrastructure.handlers.EventHandler;
import com.techbank.cqrs.core.events.BaseEvent;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.support.Acknowledgment;
import org.springframework.kafka.support.KafkaHeaders;
import org.springframework.messaging.handler.annotation.Header;
import org.springframework.messaging.handler.annotation.Payload;
import org.springframework.stereotype.Service;

import java.lang.reflect.InvocationTargetException;
```

```
@Service
public class AccountEventConsumer implements EventConsumer {
    private static final Logger logger = LoggerFactory.getLogger(AccountEventConsumer.class);

    @Autowired
    private EventHandler eventHandler;

    /**
     * Un unico listener sobre un unico topic. El JsonDeserializer reconstruye el tipo concreto
     * del evento (AccountOpenedEvent, FundsDepositedEvent, ...) a partir del header __TypeId__
     * que agrega el JsonSerializer del productor, y aqui se despacha por reflexion al metodo
     * on(...) correspondiente del EventHandler.
     */
    @KafkaListener(topics = "${spring.kafka.topic}", groupId = "${spring.kafka.consumer.group-id}")
    @Override
    public void consume(@Payload BaseEvent event,
                        @Header(KafkaHeaders.RECEIVED_PARTITION_ID) int partition,
                        @Header(KafkaHeaders.OFFSET) long offset,
                        Acknowledgment ack) {
        logger.info("CONSUMED  partition={} offset={} type={} id={} version={}",
                partition, offset, event.getClass().getSimpleName(), event.getId(), event.getVersion());
        try {
            var eventHandlerMethod = eventHandler.getClass().getDeclaredMethod("on", event.getClass());
            eventHandlerMethod.setAccessible(true);
            eventHandlerMethod.invoke(eventHandler, event);
            ack.acknowledge();
        } catch (NoSuchMethodException | IllegalAccessException | InvocationTargetException e) {
            throw new RuntimeException("Error while consuming event " + event.getClass().getSimpleName(), e);
        }
    }
}
```

4.3 Cambios complementarios

f. AccountEventProducer: key del mensaje = ID de la cuenta

Cada evento se envía con el ID del agregado como key. Con una sola partición esto no cambia el resultado, pero si el topic se escala a varias particiones, Kafka asigna todos los mensajes con la misma key a la misma partición, de modo que el orden de los eventos **de cada cuenta** se mantiene. También se agregó un log por cada evento producido.

```
account.cmd/.../infrastructure/AccountEventProducer.java
@@ -3,4 +3,6 @@ package com.techbank.account.cmd.infrastructure;
 import com.techbank.cqrs.core.events.BaseEvent;
 import com.techbank.cqrs.core.producers.EventProducer;
+import org.slf4j.Logger;
+import org.slf4j.LoggerFactory;
 import org.springframework.beans.factory.annotation.Autowired;
 import org.springframework.kafka.core.KafkaTemplate;
@@ -9,4 +11,6 @@ import org.springframework.stereotype.Service;
 @Service
 public class AccountEventProducer implements EventProducer {
+    private static final Logger logger = LoggerFactory.getLogger(AccountEventProducer.class);
+
     @Autowired
     private KafkaTemplate<String, Object> kafkaTemplate;
```

```
@@ -14,5 +18,9 @@ public class AccountEventProducer implements EventProducer {
     @Override
     public void produce(String topic, BaseEvent event) {
-        this.kafkaTemplate.send(topic, event);
+        // La key del mensaje es el ID del agregado: Kafka garantiza el orden por particion,
+        // y todos los eventos con la misma key van siempre a la misma particion.
+        this.kafkaTemplate.send(topic, event.getId(), event);
+        logger.info("PRODUCED  topic={} key={} type={} version={}",
+                topic, event.getId(), event.getClass().getSimpleName(), event.getVersion());
     }
 }
```

g. Corrección de un bug en AccountEventHandler

Durante las pruebas se detectó que el handler de *FundsWithdrawnEvent* calculaba el nuevo saldo pero nunca lo asignaba a la entidad, así que los retiros no se reflejaban en el modelo de lectura. Se corrigió con una línea:

```
account.query/.../handlers/AccountEventHandler.java
@@ -47,4 +47,5 @@ public class AccountEventHandler implements EventHandler {
         var currentBalance = bankAccount.get().getBalance();
         var latestBalance = currentBalance - event.getAmount();
+        bankAccount.get().setBalance(latestBalance);
         accountRepository.save(bankAccount.get());
     }
```

h. Infraestructura

Se agregaron *Dockerfile*, *.dockerignore* y *docker-compose.yml*. El compose incluye un contenedor *kafka-init* que crea explícitamente el topic *BankAccountEvents* con **1 partición** antes de arrancar los microservicios.

5. Parte 2: Prueba de reproducción de eventos

5.1 Un solo topic en Kafka

Después del cambio, Kafka solo tiene el topic *BankAccountEvents* (además del topic interno *__consumer_offsets*), con una partición:

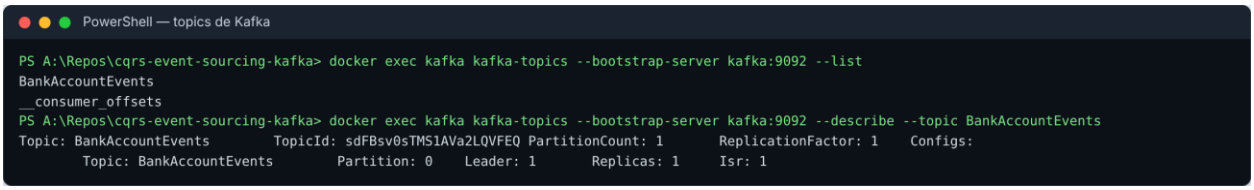


Figura 19. *Listado y descripción de los topics de Kafka.*

5.2 Serie de operaciones

Se ejecutaron operaciones sobre dos cuentas nuevas: la cuenta **C** (Carlos Ruiz) queda activa con saldo 300 + 200 − 100 + 50 = **450**, y la cuenta **D** (Ana Torres) se abre, recibe un depósito y un retiro y se cierra.

Serie de operaciones ejecutadas (Parte 2)

1	POST	Abrir cuenta C (Carlos Ruiz) http://localhost:5000/api/v1/openBankAccount body: {"accountHolder": "Carlos Ruiz", "accountType": "SAVINGS", "openingBalance": 300}	201 Created
2	PUT	Depositar 200 en C http://localhost:5000/api/v1/depositFunds/74bca510-058c-4d4d-b02b-ed4aeaceb8e8 body: {"amount": 200}	200 OK
3	PUT	Retirar 100 de C http://localhost:5000/api/v1/withdrawFunds/74bca510-058c-4d4d-b02b-ed4aeaceb8e8 body: {"amount": 100}	200 OK
4	PUT	Depositar 50 en C http://localhost:5000/api/v1/depositFunds/74bca510-058c-4d4d-b02b-ed4aeaceb8e8 body: {"amount": 50}	200 OK
5	POST	Abrir cuenta D (Ana Torres) http://localhost:5000/api/v1/openBankAccount body: {"accountHolder": "Ana Torres", "accountType": "CURRENT", "openingBalance": 800}	201 Created
6	PUT	Depositar 100 en D http://localhost:5000/api/v1/depositFunds/7e6e1307-672f-4152-970d-b5385d2ac69d body: {"amount": 100}	200 OK
7	PUT	Retirar 300 de D http://localhost:5000/api/v1/withdrawFunds/7e6e1307-672f-4152-970d-b5385d2ac69d body: {"amount": 300}	200 OK
8	DELETE	Cerrar cuenta D http://localhost:5000/api/v1/closeBankAccount/7e6e1307-672f-4152-970d-b5385d2ac69d	200 OK

Figura 20. *Operaciones ejecutadas: abrir, depositar, retirar y cerrar cuenta.*

Los logs muestran que los eventos se consumieron exactamente en el orden en que se produjeron, con offsets consecutivos (6 a 13) y versiones crecientes por agregado:

PowerShell — orden de producción y de consumo de los eventos

```
PS A:\Repos\cqrs-event-sourcing-kafka> docker logs account-cmd --since <inicio de las operaciones>
2026-10-19 20:08:32.234  INFO 1 --- [nio-5000-exec-8] c.t.a.c.i.AccountEventProducer           : PRODUCED  topic=BankAccountEvents key=74bca510-058c-4d4
d-b02b-ed4aeaceb8e8 type=AccountOpenedEvent version=0
2026-10-19 20:08:32.300  INFO 1 --- [io-5000-exec-10] c.t.a.c.i.AccountEventProducer           : PRODUCED  topic=BankAccountEvents key=74bca510-058c-4d4
d-b02b-ed4aeaceb8e8 type=FundsDepositedEvent version=1
2026-10-19 20:08:32.364  INFO 1 --- [nio-5000-exec-2] c.t.a.c.i.AccountEventProducer           : PRODUCED  topic=BankAccountEvents key=74bca510-058c-4d4
d-b02b-ed4aeaceb8e8 type=FundsWithdrawnEvent version=2
2026-10-19 20:08:32.426  INFO 1 --- [nio-5000-exec-4] c.t.a.c.i.AccountEventProducer           : PRODUCED  topic=BankAccountEvents key=74bca510-058c-4d4
d-b02b-ed4aeaceb8e8 type=FundsDepositedEvent version=3
2026-10-19 20:08:32.481  INFO 1 --- [nio-5000-exec-6] c.t.a.c.i.AccountEventProducer           : PRODUCED  topic=BankAccountEvents key=7e6e1307-672f-415
2-970d-b5385d2ac69d type=AccountOpenedEvent version=0
2026-10-19 20:08:32.549  INFO 1 --- [nio-5000-exec-8] c.t.a.c.i.AccountEventProducer           : PRODUCED  topic=BankAccountEvents key=7e6e1307-672f-415
2-970d-b5385d2ac69d type=FundsDepositedEvent version=1
2026-10-19 20:08:32.610  INFO 1 --- [io-5000-exec-10] c.t.a.c.i.AccountEventProducer           : PRODUCED  topic=BankAccountEvents key=7e6e1307-672f-415
2-970d-b5385d2ac69d type=FundsWithdrawnEvent version=2
2026-10-19 20:08:32.668  INFO 1 --- [nio-5000-exec-2] c.t.a.c.i.AccountEventProducer           : PRODUCED  topic=BankAccountEvents key=7e6e1307-672f-415
2-970d-b5385d2ac69d type=AccountClosedEvent version=3
PS A:\Repos\cqrs-event-sourcing-kafka> docker logs account-query --since <inicio de las operaciones>
2026-10-19 20:08:32.242  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer         : CONSUMED  partition=0 offset=6 type=AccountOpenedEvent
id=74bca510-058c-4d4d-b02b-ed4aeaceb8e8 version=0
2026-10-19 20:08:32.306  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer         : CONSUMED  partition=0 offset=7 type=FundsDepositedEvent
id=74bca510-058c-4d4d-b02b-ed4aeaceb8e8 version=1
2026-10-19 20:08:32.371  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer         : CONSUMED  partition=0 offset=8 type=FundsWithdrawnEvent
id=74bca510-058c-4d4d-b02b-ed4aeaceb8e8 version=2
2026-10-19 20:08:32.433  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer         : CONSUMED  partition=0 offset=9 type=FundsDepositedEvent
id=74bca510-058c-4d4d-b02b-ed4aeaceb8e8 version=3
2026-10-19 20:08:32.487  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer         : CONSUMED  partition=0 offset=10 type=AccountOpenedEvent
id=7e6e1307-672f-4152-970d-b5385d2ac69d version=0
2026-10-19 20:08:32.556  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer         : CONSUMED  partition=0 offset=11 type=FundsDepositedEven
t id=7e6e1307-672f-4152-970d-b5385d2ac69d version=1
2026-10-19 20:08:32.617  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer         : CONSUMED  partition=0 offset=12 type=FundsWithdrawnEven
t id=7e6e1307-672f-4152-970d-b5385d2ac69d version=2
2026-10-19 20:08:32.674  INFO 1 --- [ntainer#0-0-C-1] c.t.a.q.i.c.AccountEventConsumer         : CONSUMED  partition=0 offset=13 type=AccountClosedEvent
id=7e6e1307-672f-4152-970d-b5385d2ac69d version=3
```

Figura 21. *Orden de producción (account-cmd) y de consumo (account-query).*

El topic almacena todos los eventos de todas las cuentas en un único log ordenado por offset; la key de cada mensaje es el ID de la cuenta:

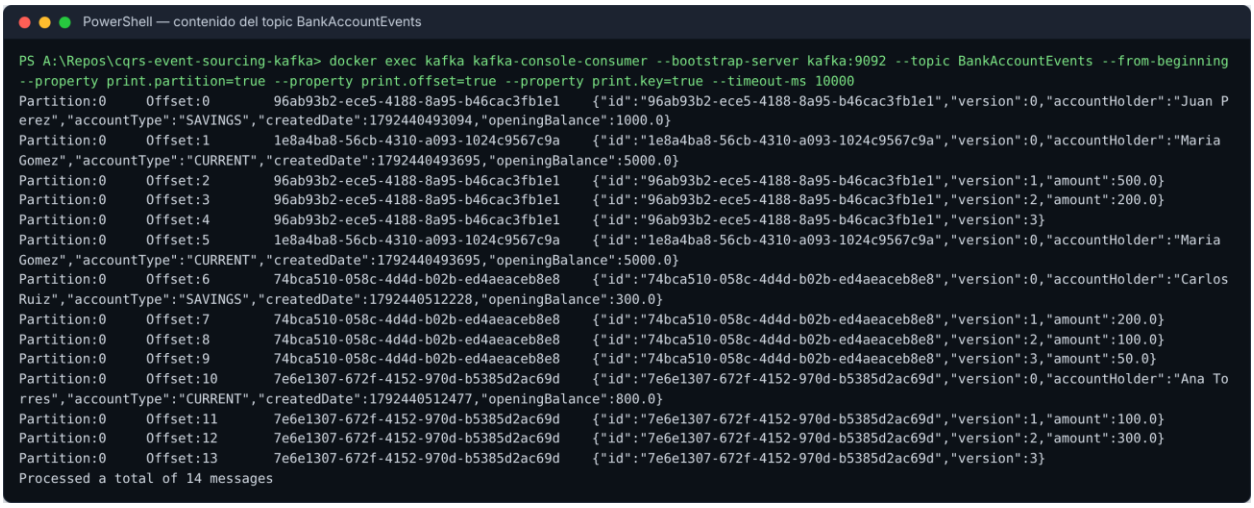


Figura 22. *Contenido completo del topic BankAccountEvents (offsets 0 a 13).*

5.3 Reinicio simple del servicio de consulta

Primero se reinició el servicio de consulta sin tocar nada más. Al arrancar, el consumer group *bankaccConsumer* retoma desde su **offset confirmado (14)**, que ya es el final del topic (LAG = 0), así que no vuelve a procesar ningún evento: hay 0 líneas CONSUMED tras el reinicio. Es el comportamiento correcto, porque el modelo de lectura ya está al día; la propiedad *auto-offset-reset: earliest* solo aplica cuando el grupo no tiene offsets confirmados.



Figura 23. *Reinicio simple: el consumidor retoma en el offset 14 y no reprocesa eventos.*

5.4 Reproducción completa de los eventos

Para observar la reproducción completa se simuló la reconstrucción del lado de lectura desde cero:

1. Se detuvo el contenedor *account-query*.
2. Se eliminó la base de datos de lectura *bankAccount* en MySQL.
3. Se reiniciaron los offsets del consumer group *bankaccConsumer* al inicio del topic (*--reset-offsets --to-earliest*).
4. Se volvió a iniciar *account-query*: Hibernate recrea la tabla (*ddl-auto: update*) y el consumidor lee desde el offset 0.

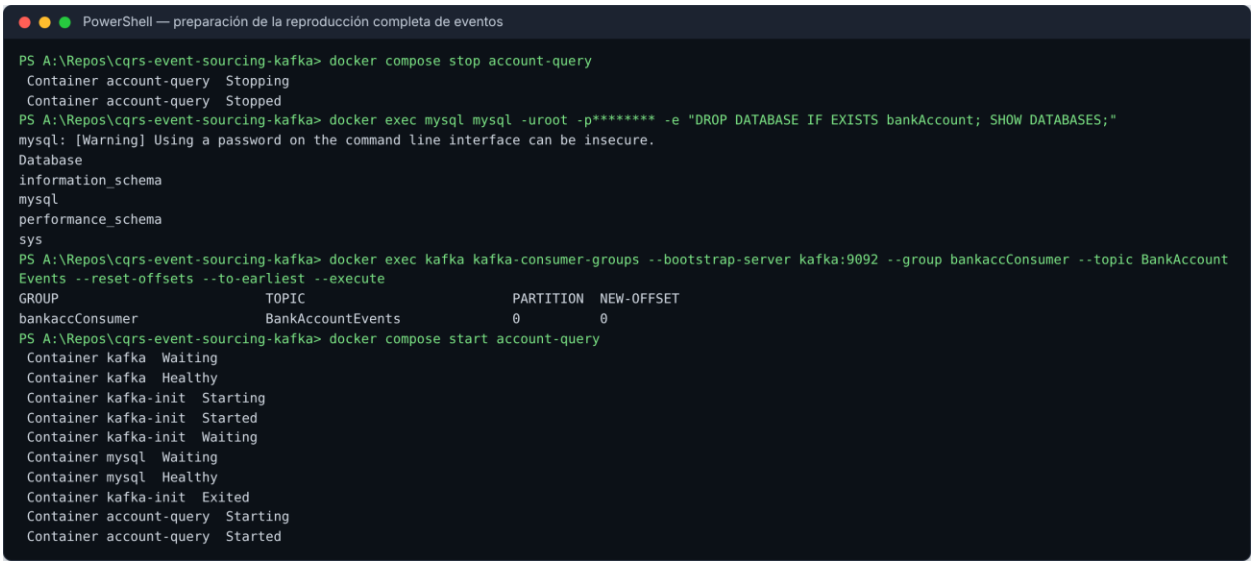


Figura 24. *Preparación de la reproducción: detener, borrar la base de lectura y reiniciar offsets.*

Los logs del servicio de consulta muestran que se reprodujeron los **14 eventos en estricto orden de offset (0 → 13)**: primero los de las cuentas A y B de la Parte 1 (incluido el *AccountOpenedEvent* republicado por el restore), luego los de C y finalmente los de D. Para cada cuenta el orden de versiones es siempre 0, 1, 2, 3: nunca se aplica un depósito antes de la apertura ni un cierre antes de un movimiento. Al terminar, el consumer group queda de nuevo en el offset 14 con LAG 0.

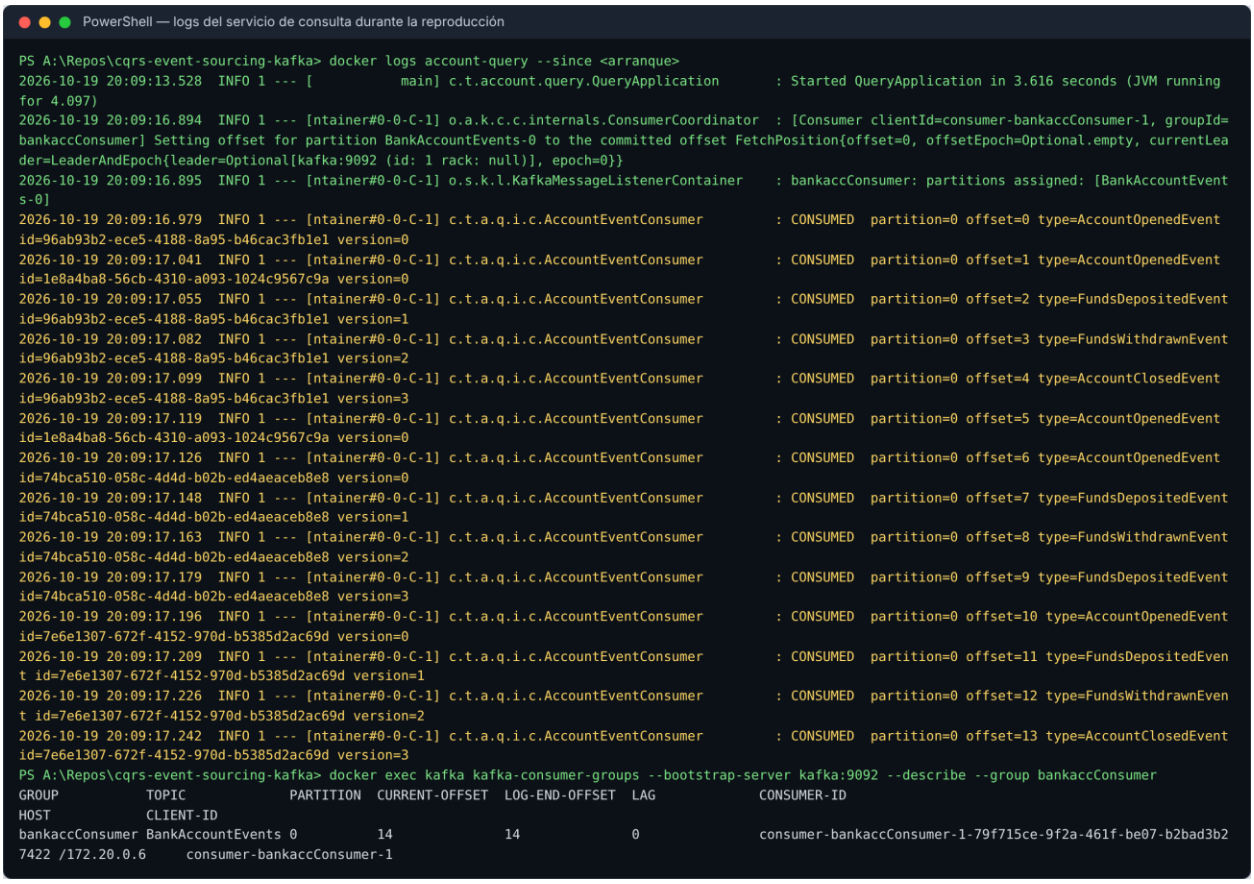


Figura 25. *Reproducción completa: eventos consumidos en orden de offset 0 a 13.*

5.5 Resultado: el modelo de lectura reconstruido es idéntico

El estado del modelo de lectura después de la reproducción es exactamente igual al que había antes:

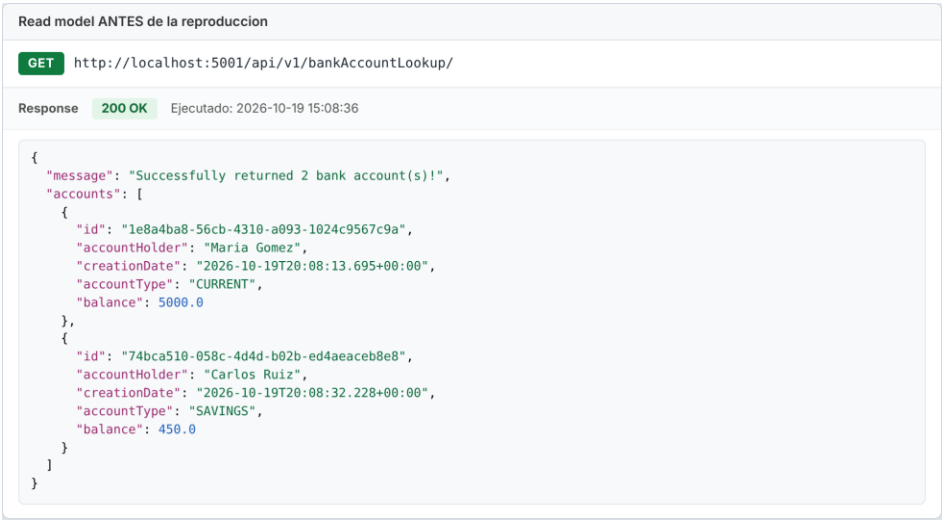


Figura 26. *Modelo de lectura ANTES de la reproducción.*

Read model DESPUES de la reproduccion

GET http://localhost:5001/api/v1/bankAccountLookup/

Response 200 OK Ejecutado: 2026-10-19 15:09:26


```
{
  "message": "Successfully returned 2 bank account(s)!",
  "accounts": [
    {
      "id": "1e8a4ba8-56cb-4310-a093-1024c9567c9a",
      "accountHolder": "Maria Gomez",
      "creationDate": "2026-10-19T20:08:13.695+00:00",
      "accountType": "CURRENT",
      "balance": 5000.0
    },
    {
      "id": "74bca510-058c-4d4d-b02b-ed4aeaceb8e8",
      "accountHolder": "Carlos Ruiz",
      "creationDate": "2026-10-19T20:08:32.228+00:00",
      "accountType": "SAVINGS",
      "balance": 450.0
    }
  ]
}
```


Figura 27. *Modelo de lectura DESPUÉS de la reproducción.*

Cuenta	Eventos (en orden)	Esperado	Después del replay
A: Juan Perez	Opened(1000) → Deposited(500) → Withdrawn(200) → Closed	Eliminada	No aparece
B: Maria Gomez	Opened(5000) → Opened(5000, republicado)	5000	5000.0
C: Carlos Ruiz	Opened(300) → Deposited(200) → Withdrawn(100) → Deposited(50)	450	450.0
D: Ana Torres	Opened(800) → Deposited(100) → Withdrawn(300) → Closed	Eliminada	204 No Content

Cuenta C despues de la reproduccion

GET http://localhost:5001/api/v1/bankAccountLookup/byId/74bca510-058c-4d4d-b02b-ed4aeaceb8e8

Response 200 OK Ejecutado: 2026-10-19 15:09:26


```
{
  "message": "Successfully returned bank account!",
  "accounts": [
    {
      "id": "74bca510-058c-4d4d-b02b-ed4aeaceb8e8",
      "accountHolder": "Carlos Ruiz",
      "creationDate": "2026-10-19T20:08:32.228+00:00",
      "accountType": "SAVINGS",
      "balance": 450.0
    }
  ]
}
```


Figura 28. *Cuenta C después de la reproducción: saldo 450.*

Cuenta D (cerrada) despues de la reproduccion

GET http://localhost:5001/api/v1/bankAccountLookup/byId/7e6e1307-672f-4152-970d-b5385d2ac69d

Response 204 No Content Ejecutado: 2026-10-19 15:09:26

(sin cuerpo — HTTP 204 No Content)

Figura 29. *Cuenta D (cerrada) después de la reproducción: 204 No Content.*

6. Cómo se garantiza el orden y por qué es importante

6.1 Cómo se garantiza

- **Un único topic y una única partición.** Kafka es un log append-only y, dentro de una partición, cada mensaje recibe un offset creciente. Todos los eventos se escriben en *BankAccountEvents*, así que su orden queda fijado por el orden de escritura.
- **Un único listener.** El servicio de consulta tiene un solo *@KafkaListener* (un solo hilo consumidor) que procesa los mensajes uno a uno en orden de offset y despacha cada evento a su handler. Ya no hay varios consumidores compitiendo en paralelo.
- **Confirmación manual del offset.** Con *ack-mode: MANUAL_IMMEDIATE*, el offset se confirma solo después de aplicar el evento al modelo de lectura; si el servicio cae, al reiniciar continúa desde el siguiente evento no confirmado, sin saltarse ninguno.
- **Key = ID del agregado.** Si el topic se particiona para escalar, todos los eventos de una misma cuenta van a la misma partición y conservan su orden relativo, que es el que importa para reconstruir el estado de cada agregado.
- **Orden de origen consistente.** En el lado de escritura, el event store asigna una versión secuencial a cada evento del agregado y aplica control de concurrencia optimista (*ConcurrencyException*); los eventos se publican en ese mismo orden.

6.2 Por qué es importante en CQRS y Event Sourcing

En Event Sourcing el estado no se guarda, se **deriva** aplicando los eventos uno tras otro, y esa operación no es conmutativa: aplicar Opened → Deposited → Closed no da el mismo resultado que Deposited → Closed → Opened. En CQRS, además, el modelo de lectura se construye de forma asíncrona a partir de esos mismos eventos; si el consumidor los aplica en un orden distinto al que se produjeron, el modelo de lectura diverge del event store de forma silenciosa: el sistema no falla, simplemente muestra saldos incorrectos.

El orden garantizado es lo que hace confiables las capacidades clave del patrón: reconstruir el modelo de lectura desde cero (como se demostró en la sección 5.4), agregar nuevas proyecciones que procesen el historial completo, recuperarse de fallos reprocesando eventos y auditar el historial. La prueba muestra que, con un solo topic, la reproducción de 14 eventos de 4 cuentas produce exactamente el mismo estado que el procesamiento original.

El costo de esta garantía es el paralelismo: una sola partición implica un solo consumidor activo por grupo. Usar el ID del agregado como key permite escalar a varias particiones manteniendo el orden por cuenta, que es el que el dominio necesita.